

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT2-Constraint-Based Problem Solving

Register Number	192511053
Name	B Santhosh Kumar

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 20%

QUESTIONS:3

Total Marks:30

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- D1 cannot work Night shift
- D2 must work before D3 (Morning < Afternoon < Night)
- Only one doctor per shift
- D3 cannot work Morning
- All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Cannot pass through obstacles
- Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

*I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Question 1: Solve the Doctor Shift Assignment Problem Using Backtracking Search

Aim:

To assign three doctors (**D1, D2, and D3**) to three hospital shifts (**Morning, Afternoon, and Night**) using the **Backtracking Search Algorithm** while satisfying all the given constraints and obtaining a valid schedule.

Introduction:

Backtracking Search is a systematic search algorithm used in Artificial Intelligence to solve **Constraint Satisfaction Problems (CSPs)**. A CSP consists of a set of variables, possible values (domains), and constraints that restrict the values that variables can take.

Backtracking constructs the solution incrementally by assigning one value at a time. After every assignment, the algorithm verifies whether the current partial solution satisfies all constraints. If any constraint is violated, the algorithm **backtracks** by removing the last assignment and tries another possible value. This process continues until a valid solution is found.

In this problem, each doctor must be assigned exactly one hospital shift while satisfying all scheduling constraints.

Problem Statement:

A hospital management system must assign **three doctors (D1, D2, and D3)** to **three different shifts**:

- Morning (M)
- Afternoon (A)
- Night (N)

The Solution must satisfy all given constraints using the **Backtracking Search Algorithm**.

Given Constraints:

Constraint No	Description
C1	D1 cannot work Night shift.
C2	D2 must work before D3 (Morning < Afternoon < Night).
C3	Only one doctor can be assigned to each shift.
C4	D3 cannot work Morning shift.
C5	Every doctor must be assigned exactly one shift.

Problem Formulation:

Component	Description
Variables	D1, D2, D3
Domain	{Morning, Afternoon, Night}
Initial State	No doctor assigned to any shift
Goal State	Assign all doctors satisfying every constraint

State Space Representation:

Each state in the search tree represents a partial or complete assignment of doctors to shifts.

State	Meaning
{}	No doctor assigned
{D1 = Morning}	Only D1 assigned
{D1 = Morning, D2 = Afternoon}	Partial assignment
{D1 = Morning, D2 = Afternoon, D3 = Night}	Complete assignment (Goal State)

The search begins with an empty assignment and gradually assigns shifts to each doctor. After every assignment, the constraints are checked before proceeding to the next doctor.

Initial State

{}

Goal State

{

D1 = Morning,

D2 = Afternoon,

D3 = Night

}

Search Strategy Used:

- The problem is solved using the **Backtracking Search Algorithm**.
- Backtracking works by assigning one variable at a time and verifying whether the current assignment satisfies all constraints. If the assignment is valid, the algorithm proceeds to assign the next variable. If any constraint is violated, the algorithm removes the last assignment (backtracks) and tries another possible value. This process continues until either a valid solution is found or all possibilities have been explored.

Algorithm (Backtracking Search):

Pseudo Code

BACKTRACK(Assignment)

If all doctors are assigned

 return Assignment

Select an unassigned doctor

For each available shift

Assign the shift

If constraints are satisfied

Continue with next doctor

Else

Remove the assignment

(Backtrack)

Return Failure if no valid assignment exists

Backtracking Search Tree:

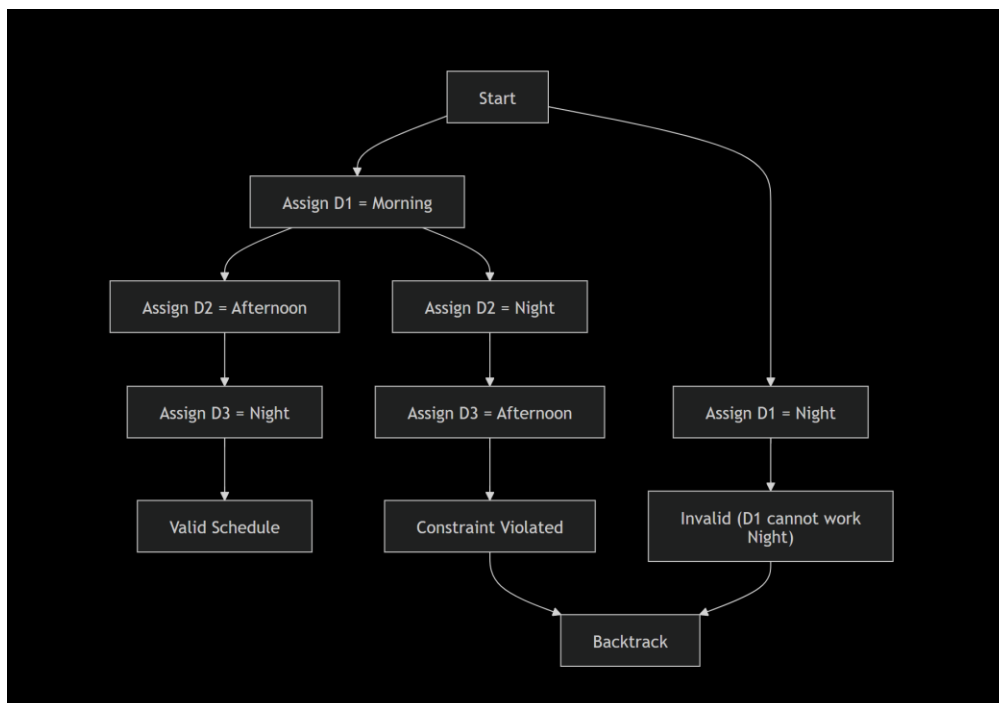


Figure 1: Backtracking Search Tree for Doctor Shift Assignment.

Diagram Credit: Created by the author using Mermaid Live Editor based on the given problem statement.

Constraint Evaluation Using Backtracking Search:

The Backtracking Search algorithm begins with an empty assignment and assigns shifts to doctors one by one. After each assignment, all applicable constraints are verified. If the assignment satisfies the constraints, the algorithm proceeds to the next doctor. Otherwise, it backtracks to the previous assignment and explores another possible shift.

Table 1: Backtracking Search Process

Iteration	Current Assignment	Constraint Verification	Decision
1	{}	Initial state. No doctor assigned.	Start Search
2	D1 → Morning	D1 is not assigned to Night. Constraint C1 satisfied.	Continue
3	D2 → Afternoon	Afternoon is available. D2 can still work before D3. Constraints C2 and C3 satisfied.	Continue
4	D3 → Night	D3 is not assigned Morning. Afternoon occurs before Night. All doctors have unique shifts.	Valid Solution Found
5 (<i>Alternative Trial</i>)	D1 → Morning, D2 → Night	Partial assignment is temporarily accepted.	Continue
6 (<i>Alternative Trial</i>)	D3 → Afternoon	D2 is working after D3. Constraint C2 is violated.	Reject Assignment
7	Remove D3 = Afternoon	Previous assignment is discarded.	Backtrack
8	Assign D2 = Afternoon	Constraint satisfied.	Continue
9	Assign D3 = Night	All constraints satisfied.	Goal State Reached

Backtracking Illustration:

The following assignment does **not** satisfy the constraints.

Attempt 1

Doctor	Shift
D1	Morning
D2	Night
D3	Afternoon

Checking Constraint C2:

D2 must work before D3

However,

Night comes after Afternoon

Therefore,

Constraint Violated

The algorithm immediately rejects this assignment and returns to the previous valid state.

This process is called **Backtracking** because the search moves backward whenever an invalid assignment is encountered.

Final Doctor Shift Schedule

After exploring different assignments and eliminating invalid combinations, the Backtracking Search algorithm produces the following schedule.

Table 2: Final Shift Allocation

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Night	D3

Search Sequence:

Start

|



Assign D1 → Morning

|



Assign D2 → Afternoon

|



Assign D3 → Night

|



Check All Constraints

|



Goal State Achieved

Result:

The Backtracking Search algorithm successfully found a valid doctor shift schedule by assigning one shift to each doctor while satisfying all the given constraints. Invalid assignments were rejected through backtracking, and the final schedule ensured that each doctor was assigned exactly one shift without any conflicts.

Question 2

Robot Path Planning Using Breadth-First Search (BFS)

Aim:

To determine the shortest path for a robot moving in a 5×5 grid environment from the Start position (S) to the Goal position (G) using the Breadth-First Search (BFS) algorithm while avoiding obstacles and calculating the path cost.

Introduction:

Autonomous robots frequently operate in environments where they must travel from one location to another while avoiding obstacles. Path planning enables the robot to identify a safe route using the available information. Artificial Intelligence provides several search algorithms for this purpose, among which Breadth-First Search (BFS) is one of the simplest and most reliable methods for finding the shortest path when every movement has the same cost.

BFS explores the search space level by level. It first visits all neighbouring cells before moving to the next level of the search tree. Since every movement in this problem has a uniform cost of one, BFS guarantees the shortest path from the starting point to the goal whenever a path exists.

Because the grid configuration is not provided in the question, a suitable 5×5 environment is assumed to demonstrate the complete working of the BFS algorithm.

Problem Statement:

A robot is placed in a 5×5 **grid environment**. The objective is to move from the **Start (S)** position to the **Goal (G)** position while avoiding blocked cells.

The robot may move only in the four cardinal directions and every movement has the same cost.

The Breadth-First Search algorithm is used to determine the shortest path

Assumption:

Since the grid layout is not specified in the question, the following grid is assumed.

5 × 5 Grid Environment

S			X	
	X		X	
	X			
		X		
X				G

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- Blank cell = Free space

Figure:1 Assumed 5 × 5 Grid Environment

Figure Credit: Created by the author based on the given problem statement.

Given Constraints:

Constraint	Description
C1	Robot can move Up, Down, Left and Right only.
C2	Cost of every movement is 1.
C3	Robot cannot move through obstacle cells.
C4	Manhattan Distance is calculated as the heuristic value.

Problem Formulation:

Component	Description
State	Robot position (row, column)
Initial State	$S = (1,1)$
Goal State	$G = (5,5)$
Operators	Up, Down, Left, Right
Path Cost	1 for every movement

Algorithm Used:

Breadth-First Search (BFS)

Begin

Create an empty Queue

Insert Start node

While Queue is not empty

Remove first node

If Goal reached

Stop

Else

Expand all valid neighbours

Insert unexplored neighbours into Queue

Repeat

End

BFS Search Tree:

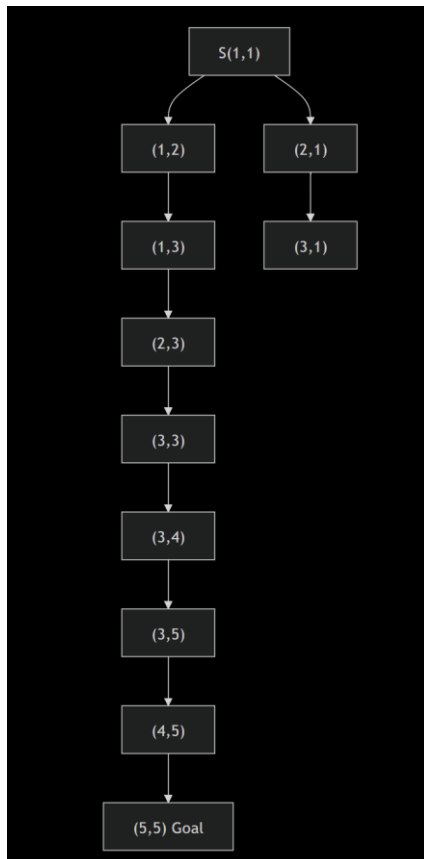


Figure 2 : BFS Search Tree

Figure Credit: Created using Mermaid Live Editor

Breadth-First Search Execution:

Step	Expanded Node	Queue After Expansion
1	S (1,1)	(1,2), (2,1)
2	(1,2)	(2,1), (1,3)
3	(2,1)	(1,3), (3,1)
4	(1,3)	(3,1), (2,3)
5	(3,1)	(2,3), (4,1)
6	(2,3)	(4,1), (3,3)
7	(3,3)	(4,1), (3,4)
8	(3,4)	(4,1), (3,5)
9	(3,5)	(4,5)
10	(4,5)	(5,5)
11	(5,5)	Goal Found

Manhattan Distance Calculation:

The Manhattan Distance is

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

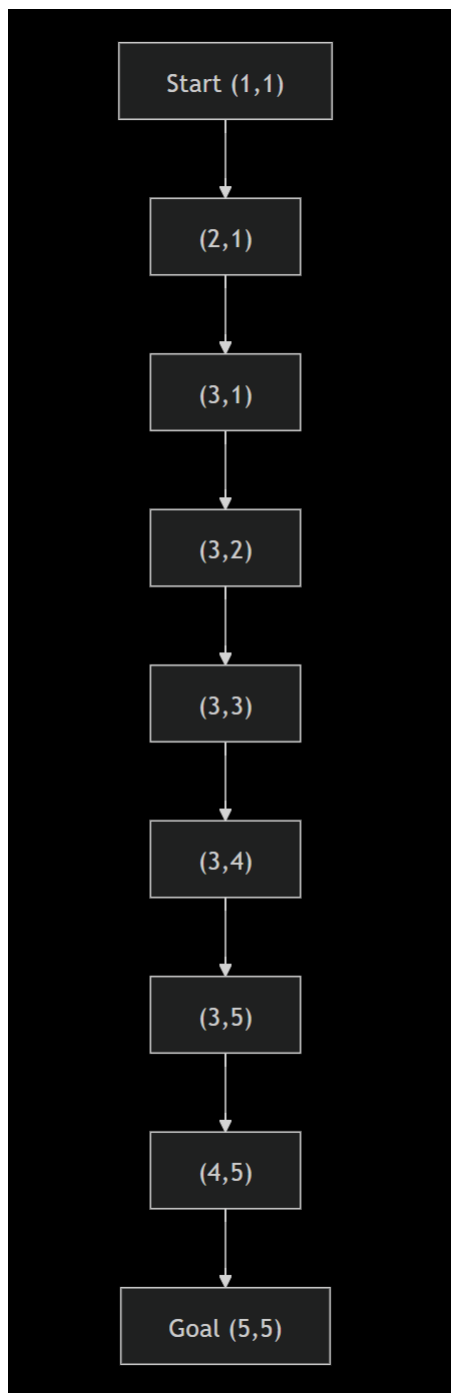
Example

For Start (1,1)

Goal (5,5)

$$h = (5 - 1) + (5 - 1) = 8$$

Optimal Path



Total Cost:

Every movement costs **1**.

Number of movements = **8**

Therefore,

Total Path Cost = 8

Result:

The Breadth-First Search algorithm expands nodes level by level and guarantees the shortest path because every movement has an equal cost. Obstacle cells are ignored during expansion, ensuring that the robot travels only through valid locations. Although Manhattan Distance is calculated as specified in the question, it is not used to determine the expansion order in BFS.

Question 3

Autonomous Rescue Robot in a Disaster-Affected Building

Aim:

To analyze the working of an autonomous rescue robot in a disaster-affected building and determine a safe and minimum-cost path to rescue a survivor using an appropriate Artificial Intelligence search strategy.

Introduction:

Autonomous rescue robots are used in disaster management to search for survivors in dangerous areas where human rescue operations are difficult. These robots can move inside damaged buildings, identify blocked paths, and reach survivors safely. Since the robot cannot observe the complete environment, it has to make decisions based on the information available at each step.

Artificial Intelligence helps the robot plan its movement by selecting safe paths and avoiding obstacles. In this problem, the robot operates in a **partially observable environment**, where only the adjacent rooms can be sensed. Therefore, the robot updates its knowledge while moving. To find a safe path with minimum travelling cost, an **Online Search Agent** together with **Uniform Cost Search (UCS)** is used.

Problem Statement:

An autonomous rescue robot is deployed inside a 5×5 **building** represented as a grid. The robot starts from **Start (S)** and must reach the survivor located at **Goal (G)**.

The robot can move only in four directions: Up, Down, Left and Right. Some rooms are blocked by obstacles, while some rooms are risky and increase the travelling cost. Since the robot can observe only the neighbouring rooms, it must continuously update its knowledge and choose the safest path to rescue the survivor.

Assumptions:

Since the grid is not specified in the question, the following assumptions are considered.

- The building is represented by a 5×5 **grid**.
- The robot starts from **S**.

- The survivor is located at **G**.
- **X** represents blocked rooms.
- **R** represents risky zones.
- Blank cells indicate safe rooms.
- The robot can move only one cell at a time.
- The robot updates its map whenever a new room is explored.

Figure 1: Assumed Disaster Building Layout

S		R	X	
	X		X	
	R			
		X	R	
X				G

Figure Credit: Created by the author based on the given problem statement.

Given Constraints:

The rescue robot should satisfy the following constraints during navigation.

- The robot can move only **Up, Down, Left and Right**.
- Obstacle cells cannot be crossed.
- The robot can observe only the adjacent rooms.
- Every normal movement has a cost of **1**.
- Entering a risky room increases the cost by **2**.
- The robot should minimize the total travelling cost.
- The robot must update its knowledge dynamically while exploring.

Problem Formulation:

Component	Description
Initial State	Robot at Start (S)
Goal State	Reach Survivor (G)
State Representation	Robot position (Row, Column)
Operators	Up, Down, Left, Right
Search Strategy	Online Search with Uniform Cost Search
Path Cost	Normal Move = 1, Risk Zone = 3

Online Search Process:

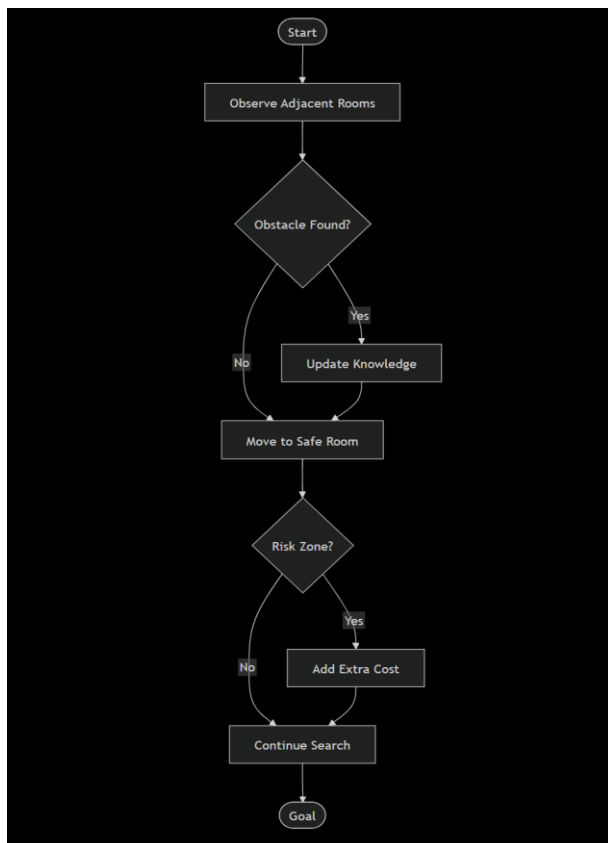


Figure 2: Online Search Process of the Rescue Robot.

Figure Credit: Created using Mermaid Live Editor.

(a) Analysis of the Given Constraints

The rescue robot must satisfy all the given constraints while searching for the survivor. Each constraint affects the robot's navigation and decision-making process. The analysis is given below.

Constraint	Effect on Robot
Robot can move Up, Down, Left and Right	The robot can move only to adjacent rooms. Diagonal movement is not allowed.
Obstacles cannot be traversed	Blocked rooms are ignored and an alternative path is selected.
Limited sensing capability	The robot can observe only neighbouring rooms and cannot view the complete building at once.
Movement cost = 1	Every normal movement increases the total path cost by one.
Risk zone adds cost = 2	The robot calculates the additional cost before entering a risky room.
Minimize total cost	The robot always prefers the path with the least accumulated cost.
Dynamic knowledge update	Newly discovered obstacles and risky zones are stored in memory for future decisions.

From the above analysis, it is clear that the robot should not only reach the goal but also avoid obstacles, reduce travelling cost, and update its knowledge continuously.

Robot Decision Process:

The rescue robot follows the steps given below while moving towards the survivor.

Step 1: The robot starts from the initial position S.

Step 2: It observes all neighbouring rooms using its sensors.

Step 3: If an obstacle is detected, that path is rejected and another path is explored.

Step 4: If a risky zone is detected, the additional cost is calculated before deciding whether to enter.

Step 5: The robot updates its internal map with the newly discovered information.

Step 6: Using Uniform Cost Search, the robot selects the path having the minimum total cost.

Step 7: The search continues until the survivor is reached.

(b) Solution Using Uniform Cost Search

Uniform Cost Search (UCS) expands the node having the **lowest accumulated cost**. Since risky rooms increase the travelling cost, UCS compares all possible paths and selects the safest minimum-cost route.

The Online Search Agent helps the robot update its knowledge whenever new information becomes available.

Step-by-Step Robot Navigation

Step	Current Position	Observation	Robot Action	Total Cost
1	S (1,1)	Starting Position	Begin Search	0
2	(1,2)	Safe Room	Move Right	1
3	(2,2)	Obstacle Found	Select Another Path	1
4	(2,3)	Safe Room	Continue Search	2
5	(3,3)	Safe Room	Move Forward	3
6	(3,4)	Risk Zone Nearby	Choose Safer Route	4
7	(4,5)	Goal Nearby	Move Forward	5
8	G (5,5)	Survivor Found	Rescue Completed	6

Path Cost Calculation:

Path Component	Cost
Normal Movements	6
Risk Zones Entered	0
Additional Risk Cost	0
Total Cost	6

Robot Navigation Path:

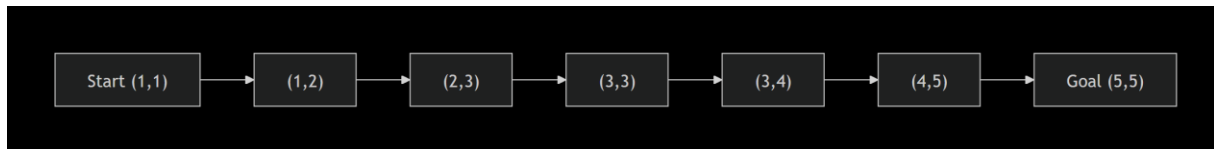


Figure 3: Navigation path followed by the rescue robot from the Start position to the Goal.

Figure Credit: Created using Mermaid Live Editor.

(c) Application of Artificial Intelligence Concepts

The rescue robot applies the following Artificial Intelligence concepts to complete the rescue operation efficiently.

Intelligent Agent:

An intelligent agent is a system that observes its environment, makes decisions, and performs suitable actions to achieve a goal. In this problem, the rescue robot acts as an intelligent agent because it senses the surrounding environment, identifies obstacles and risky areas, updates its knowledge, and moves towards the survivor.

PEAS Representation:

Component	Description
Performance Measure	Reach the survivor safely with minimum travelling cost
Environment	Disaster-affected building (5 × 5 Grid)
Actuators	Robot wheels and movement controller
Sensors	Camera, Distance Sensor, Obstacle Sensor

Rationality:

A rational agent always selects the action that produces the best result based on the available information. In this problem, the robot behaves rationally by avoiding blocked paths, comparing travelling costs, and selecting the safest path to reach the survivor.

Environment Type:

The operating environment of the rescue robot is classified as follows.

Environment Property	Type
Observability	Partially Observable
Number of Agents	Single Agent
Nature	Dynamic
Decision Process	Sequential
State Space	Discrete

The robot cannot observe the complete environment initially; therefore, it continuously updates its knowledge while exploring the building.

Intelligent Agent Architecture:

Sensors

|



Rescue Robot

|

Decision Making

|



Action

|



Environment

(d) Justification of the Search Strategy

The combination of **Online Search** and **Uniform Cost Search** is suitable for this problem because:

- The robot does not know the complete environment initially.
- New obstacles are identified only while moving.
- The robot updates its knowledge dynamically.
- Uniform Cost Search always selects the path with the lowest accumulated cost.

Result:

The autonomous rescue robot successfully reaches the survivor by using an **Online Search Agent** with **Uniform Cost Search**. During navigation, the robot observes neighbouring rooms, avoids blocked paths, updates its knowledge dynamically, and selects the minimum-cost route. The solution satisfies all the given constraints and demonstrates the effective application of Artificial Intelligence in autonomous rescue operations.